

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour

Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _S.DIVYASRI(192524388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _S.DIVYASRI

Question 1 — AI-Powered Drone Delivery in a Flood-Affected Region

Scenario Understanding

The scenario places an autonomous delivery drone in a disaster-response environment where the terrain, weather, and accessibility of routes change continuously. The drone must reach flood victims with emergency medicine as quickly as possible while managing incomplete and evolving information about the state of the road network. This is fundamentally a real-time pathfinding problem under uncertainty, and the choice of search strategy directly affects the safety, timeliness, and reliability of the delivery.

i. Behaviour of Greedy Best-First Search

Greedy Best-First Search selects the next node to expand purely on the basis of the heuristic function $h(n)$, the straight-line distance to the flood-affected destination, without considering the actual path cost $g(n)$ already incurred. In this scenario, the drone would always move toward the node that appears geographically closest to the target, ignoring whether that direction is currently blocked, waterlogged, or associated with severe weather.

Strengths: the algorithm is computationally light and expands very few nodes, which suits the drone's need for rapid decisions under time pressure. It converges quickly toward the goal direction when the heuristic is broadly reliable.

Weaknesses under uncertainty: because it disregards accumulated cost, Greedy Best-First Search can be lured toward a route that looks short in straight-line terms but turns out to be blocked, flooded, or far more expensive once real terrain and weather are factored in. It is neither complete nor optimal in a dynamic environment; a single misleading heuristic estimate can cause it to commit to a route that later requires costly backtracking, which is dangerous when medicine must arrive within a strict time window.

ii. Behaviour of A* Search

A* evaluates every candidate node using $f(n) = g(n) + h(n)$, combining the real cost accumulated so far (distance already flown, terrain difficulty, weather penalty) with the heuristic estimate of the remaining distance. This balances two competing pressures: the drone is not simply attracted to

the node nearest the destination, but to the node that offers the best trade-off between confirmed cost and estimated remaining cost.

In the flood scenario, A* would naturally favour a slightly longer route that avoids a high-cost, weather-affected segment over a shorter route that looks attractive on the map but carries a high real-world traversal cost. Provided the heuristic is admissible (never overestimates true cost), A* guarantees an optimal path, which is valuable when 'optimal' corresponds to minimum delay and minimum risk to the cargo.

iii. Impact of Heuristic Accuracy

Aspect	Greedy Best-First Search	A* Search
Accurate heuristic	Performs efficiently and often finds a reasonable path quickly.	Performs efficiently and is guaranteed optimal; fewer nodes expanded.
Inaccurate / misleading heuristic	Can be led far off course, since cost-so-far is never checked; may fail to find a safe route at all.	May expand more nodes and lose efficiency, but optimality is preserved as long as the heuristic remains admissible.
Overestimating heuristic	Greedy behaviour becomes even more short-sighted.	Optimality guarantee is lost; A* degrades toward greedy-like behaviour.

In short, Greedy Best-First Search is highly sensitive to heuristic quality because it has no mechanism to compensate for a poor estimate, whereas A* is more robust because the $g(n)$ term continually corrects the search using real, observed cost.

iv. Effect of Dynamic Changes (Blocked Paths)

Both algorithms are formulated for essentially static search spaces, so a blocked path discovered mid-flight forces replanning. For Greedy Best-First Search, a blockage on the 'closest-looking' route simply causes the drone to search again from scratch with no memory of cost already spent, which wastes time and fuel. For A*, a blockage still requires replanning, but because A* already tracks accumulated cost, it can be adapted more gracefully using incremental variants such as D* or D* Lite, which reuse prior search effort and update only the affected part of the path rather than restarting entirely. This makes A*-based approaches considerably more resilient to the kind of partial, real-time updates described in the scenario.

v. Recommendation and Justification

A* search (or a real-time/incremental variant such as D* Lite) is the more suitable choice for this emergency-delivery scenario. While Greedy Best-First Search offers speed, its disregard for actual traversal cost makes it unsafe when routes carry variable, weather-dependent risk. A* balances responsiveness with correctness: it uses the heuristic to stay fast while the cost term keeps it grounded in real, observed conditions, and it degrades gracefully when combined with replanning techniques for blocked paths. Given that the priorities stated in the scenario are minimum delay and minimum risk simultaneously, A*'s cost-aware optimality is the deciding factor over Greedy Best-First Search's pure speed.

Question 2 — Smart-City Traffic Signal Optimization

Formulation as an Optimization Problem

This problem is best formulated as a continuous, large-scale optimization problem rather than a classical single-path search problem. The state is a vector of signal-timing configurations across hundreds of intersections; the objective function combines total waiting time, fuel consumption, and congestion level, typically as a weighted sum to be minimised. Traditional uninformed or informed tree/graph search (e.g., BFS, A*) is inefficient here because the search space grows combinatorially with the number of intersections and possible timing values, there is no single well-defined 'goal state' to search toward, and the environment changes continuously as traffic flows in real time — properties that suit local, iterative optimisation methods far better than path-based search.

i. Hill Climbing in This Scenario

Hill Climbing would start from a candidate signal-timing configuration and iteratively move to a neighbouring configuration only if it improves the objective (e.g., reduces average waiting time), stopping when no neighbouring move improves the outcome. It is simple, fast, and memory-efficient, which is attractive for a system that must react quickly.

Limitations: Hill Climbing only accepts improving moves, so it has no way to escape a configuration that is locally good but globally poor. It also has no model of the future — it optimises the immediate objective without anticipating how traffic patterns will shift, so a configuration that is optimal at 9 a.m. may quickly become poor by 9:15 a.m. without the algorithm actively correcting for it.

ii. Local Maxima, Plateaus and Ridges

Issue	Technical Meaning	Real-World Interpretation in Traffic Control
Local maxima	A configuration where every neighbouring change makes the objective worse, but a better configuration exists elsewhere.	A set of signal timings that seems best for a cluster of intersections but ignores a superior city-wide pattern that would require several intersections to change simultaneously.

Issue	Technical Meaning	Real-World Interpretation in Traffic Control
Plateaus	A flat region where neighbouring configurations yield nearly identical objective values.	Several timing patterns that all produce roughly the same waiting time, so the algorithm 'wanders' without clear progress, wasting computation during peak hours.
Ridges	A narrow path of improvement that ordinary single-step moves cannot follow directly.	An improvement that requires coordinated changes at several adjacent intersections at once (e.g., a 'green wave'); changing one signal at a time never appears to help.

iii. Overcoming These Limitations

Simulated Annealing addresses the local-maxima problem by occasionally accepting a worse configuration with a probability that decreases over time (governed by a 'temperature' schedule). This lets the system escape local optima early in the search while converging toward good solutions as the temperature cools, which suits traffic optimisation since early exploration can discover city-wide coordination patterns before settling into fine-tuned timings.

Genetic Algorithms address the same problem differently: a population of candidate timing plans is evolved through selection, crossover, and mutation across generations. Because many candidate solutions are explored in parallel, Genetic Algorithms are naturally suited to escaping plateaus and discovering ridge-like coordinated improvements (such as green-wave sequencing across adjacent intersections), and they scale well to the very large, high-dimensional search space of a city-wide signal network.

iv. Recommended Approach

A Genetic Algorithm is the more suitable overall approach for this scenario. Its population-based search is well matched to a very large, multi-intersection search space, it inherently explores multiple regions in parallel (reducing the risk of premature convergence to a local maximum), and it adapts naturally to changing traffic patterns by re-running or continuously evolving the population as new data arrives. Simulated Annealing remains a strong, lighter-weight alternative for smaller sub-regions of the city or for fine-tuning a solution that a Genetic Algorithm has already found, so a hybrid strategy — Genetic Algorithms for global, city-wide coordination.

Question 3 — Autonomous Mars Rover Exploration

Scenario Understanding

The rover operates on unknown, unmapped Martian terrain with limited sensors and no prior complete map. It must make navigation decisions as it perceives its surroundings, updating its understanding of the environment continuously. This setting is characteristic of an online search problem rather than a classical offline search problem.

i. Why an Online Search Agent Is Required

Offline search assumes the agent has complete knowledge of the state space in advance and can compute an entire solution path before acting. The rover cannot make this assumption: the terrain map is unknown until sensed, hazards such as craters and rocks are only discovered locally, and communication delay with Earth rules out constant external replanning. An online search agent interleaves computation with action — it takes a step, senses the environment, updates its knowledge, and decides the next action — which is the only viable approach when the full state space cannot be known beforehand.

ii. Challenges of Partial Observability and Dynamic Environments

- The rover only perceives its immediate surroundings, so it cannot rule out hazards beyond current sensor range, increasing the risk of costly or unsafe moves.
- Sensor noise and limited field of view mean the rover's belief about the terrain may be incomplete or inaccurate, requiring probabilistic reasoning rather than certainty-based planning.
- Because the environment is only revealed incrementally, earlier decisions may turn out to be sub-optimal once new information (e.g., a hidden crater just beyond a ridge) becomes available, and the rover cannot always undo an action already taken.
- Limited onboard computation and power constrain how much re-planning can be performed at each step, forcing a trade-off between thoroughness and responsiveness.

iii. Building and Updating the Internal Model

As the rover explores, it incrementally constructs an internal representation of the terrain — commonly an occupancy grid or topological map — marking cells as free, hazardous, or unknown based on sensor readings. Each new observation updates this internal model, refining previously uncertain regions and revealing previously unknown ones. This is analogous to the agent maintaining and updating a belief state, and it allows the rover to reuse knowledge from earlier in the mission rather than treating every step as an isolated decision.

iv. Online Search versus Uninformed and Informed Search

Property	Uninformed Search (e.g., BFS/DFS)	Informed Search (e.g., A*)	Online Search
Map knowledge required	Complete, known in advance	Complete, known in advance	Built incrementally as the agent explores
Use of heuristics	None	Yes, guides search efficiently	Optional; can combine with heuristics as terrain becomes known
Suitability for unknown terrain	Poor — cannot plan without a map	Poor — heuristic needs a model of the space	Well suited — designed for unknown, evolving environments
Action-computation relationship	Plan fully, then act	Plan fully, then act	Interleaves acting and planning

v. Justification of the Most Suitable Approach

An online, informed search strategy — for example, an online variant of A* combined with incremental replanning (such as LRTA*, Learning Real-Time A*, or D*-style methods) — is the most appropriate choice. It provides the responsiveness required for an environment that can only be discovered through action (unlike offline uninformed or informed search), while still exploiting available heuristic information, such as estimated distance to a target sample site, once partial knowledge of the terrain exists. This combination directly addresses the scenario's three priorities: it handles uncertainty by continuously updating its internal model, it adapts to newly discovered hazards without needing a full map in advance, and it improves safety by allowing the rover to avoid committing irreversibly to a path before hazards are confirmed.

Question 4 — University Exam Timetabling as a CSP

Scenario Understanding

Automatic exam timetabling is a natural Constraint Satisfaction Problem (CSP): the system must assign values (time slots, halls) to variables (exams) such that a set of hard constraints — no student conflicts, hall capacity, precedence, faculty availability, and special accommodations — is satisfied simultaneously across potentially thousands of exams.

i. Variables, Domains, and Constraints

Component	Definition in This Scenario
Variables	Each variable represents one exam to be scheduled, e.g., Exam(CSA17), Exam(MEC21), etc.
Domains	For each exam, the domain is the set of feasible (time slot, hall) pairs, e.g., {Mon-9am-HallA, Mon-9am-HallB, Tue-2pm-HallA, ...}.
Unary constraints	Restrict a single exam's domain directly, e.g., an exam requiring a lab must only be assigned lab-equipped halls; an exam with accommodation needs must be assigned a hall with the required facilities.
Binary constraints	Relate pairs of exams, e.g., two exams taken by any common student cannot share the same time slot (no overlapping exams); a prerequisite exam must be scheduled before its dependent exam (precedence).
Resource constraints	Bound how many exams can use a shared resource at once, e.g., the number of exams scheduled in the same slot cannot exceed the number of available halls, and no faculty member can be assigned to invigilate two simultaneous exams.
Global/complex constraints	Combine several exams and resources at once, e.g., all exams for a given student cohort must fit within an examination window without exceeding a maximum number of exams per day.

ii. Backtracking Search in This Scenario

Backtracking search assigns exams to time-slot/hall pairs one at a time, checking after each assignment whether all constraints involving already-assigned exams remain satisfied. If a partial assignment violates a constraint — for instance, scheduling two exams with an overlapping student in the same slot — the algorithm backtracks and tries a different value for the most recently assigned variable (or an earlier one) rather than exploring the entire timetable space exhaustively.

Ordering heuristics are important in practice: variables with the fewest remaining valid options (minimum-remaining-values) are typically assigned first, since they are most likely to cause failures and are cheaper to detect early.

iii. Constraint Propagation and Forward Checking

Constraint propagation reduces the search space before or during backtracking by removing values from a variable's domain that can no longer be consistent given current assignments. Forward checking, applied after each exam is scheduled, immediately removes any (time slot, hall) options from the domains of related exams that would now violate a constraint — for example, once one exam is fixed to Monday 9 a.m. in Hall A, forward checking removes 'Monday 9 a.m.' from the domains of every exam sharing a student with it. This detects dead ends far earlier than backtracking alone, considerably reducing wasted search effort in a problem of this scale.

iv. Real-World Challenges and Strategy for Scalability

- As the number of courses, students, and halls grows, the constraint graph becomes extremely dense, and pure backtracking without propagation becomes computationally impractical.
- Some constraints (special accommodations, faculty preferences) are soft rather than hard, requiring the problem to be treated partly as constraint optimisation rather than pure satisfaction.
- Real institutions need fast re-scheduling when late changes occur (e.g., a hall becomes unavailable), which favours incremental, locally-repairing techniques over full re-solution.

For scalability, the most suitable strategy combines backtracking search with strong constraint propagation (forward checking and, where feasible, arc-consistency such as AC-3) together with intelligent variable- and value-ordering heuristics (minimum-remaining-values and least-constraining-value). For very large university-wide timetables, this CSP-based core is often supplemented with local-search repair methods (e.g., min-conflicts) to refine an initial feasible schedule quickly, and with soft-constraint optimisation to balance competing preferences such as faculty availability. This combination gives both the correctness guarantees CSP techniques provide for hard constraints and the scalability needed for real institutional timetables.

Question 5 — Strategic Game AI: Minimax and Alpha-Beta Pruning

Scenario Understanding

The gaming company needs an AI opponent that reasons about an adversarial, real-time environment: it must anticipate a human opponent's best responses, evaluate a very large space of possible future game states, and produce a decision within a strict time budget.

i. How Minimax Models the Problem

Minimax represents the game as a tree in which alternating levels correspond to the AI's moves (the maximising player, seeking the highest evaluation score) and the opponent's moves (the minimising player, seeking the lowest score for the AI). The algorithm recursively evaluates the tree from the leaves upward, assuming the opponent always plays optimally against the AI, and propagates the best guaranteed outcome back to the root. This directly models the adversarial nature of the game: the AI does not simply choose the move that looks best immediately, but the move that remains best after accounting for the opponent's best possible counter-play.

ii. Computational Challenges of Large Game Trees

The size of a game tree grows exponentially with search depth — for branching factor b and depth d , the tree contains roughly b^d nodes. In a real-time strategy game with many possible actions per turn, this makes full-depth Minimax search computationally infeasible within the time limits available, especially since real-time games often do not allow the luxury of long 'thinking' pauses. Memory usage, the cost of evaluating each leaf state, and the need to repeat this process for every decision the AI makes during play all compound the problem.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning is an optimisation of Minimax that maintains two bounds while traversing the tree: alpha, the best value the maximising player can guarantee so far, and beta, the best value the minimising player can guarantee so far. Whenever a branch is found where the minimising player could force a result worse than the maximiser's current alpha (or vice versa), that branch can be pruned — the search skips it entirely because a rational opponent would never allow the game to reach it.

This does not change the final decision Minimax would reach, but it can, with good move ordering, reduce the effective branching factor from b to approximately the square root of b , allowing the search to reach roughly twice the depth in the same time budget. In practice, this is what makes Minimax-based decision-making viable for the large trees typical of strategy games.

iv. Designing an Evaluation Function

Since full-depth search to terminal states is rarely possible in real time, the AI must cut the search off at a limited depth and score the resulting non-terminal states using a heuristic evaluation function. A reasonable evaluation function for a real-time strategy game would combine several weighted factors:

Factor	Justification
Material / unit advantage	The relative strength and number of units or resources each side controls is a strong direct indicator of who is winning.
Positional strength	Board or map control, terrain advantage, and proximity to key objectives reflect strategic advantage beyond raw unit count.
Economic output	Resource-generation rate captures long-term potential, which matters heavily in strategy games where the game state evolves over many turns.
Threat and safety assessment	Immediate tactical threats (units in danger, incoming attacks) must be weighted heavily since ignoring them can cause rapid, irreversible losses.
Tempo / mobility	The number of available productive actions reflects flexibility and initiative, which often predicts future advantage even when the current material score is even.

These factors are combined as a weighted sum, with weights tuned (e.g., through self-play or machine learning) so the evaluation correlates well with actual game outcomes.

v. Balancing Optimality and Real-Time Performance

Given the strict time constraints described in the scenario, the recommended strategy is Alpha-Beta pruning combined with iterative deepening: the AI searches to a shallow depth first, keeps the best move found so far, and progressively deepens the search until the time budget is exhausted, at which point it returns the best move discovered. Combined with good move ordering (searching likely-best moves first, which improves pruning efficiency) .